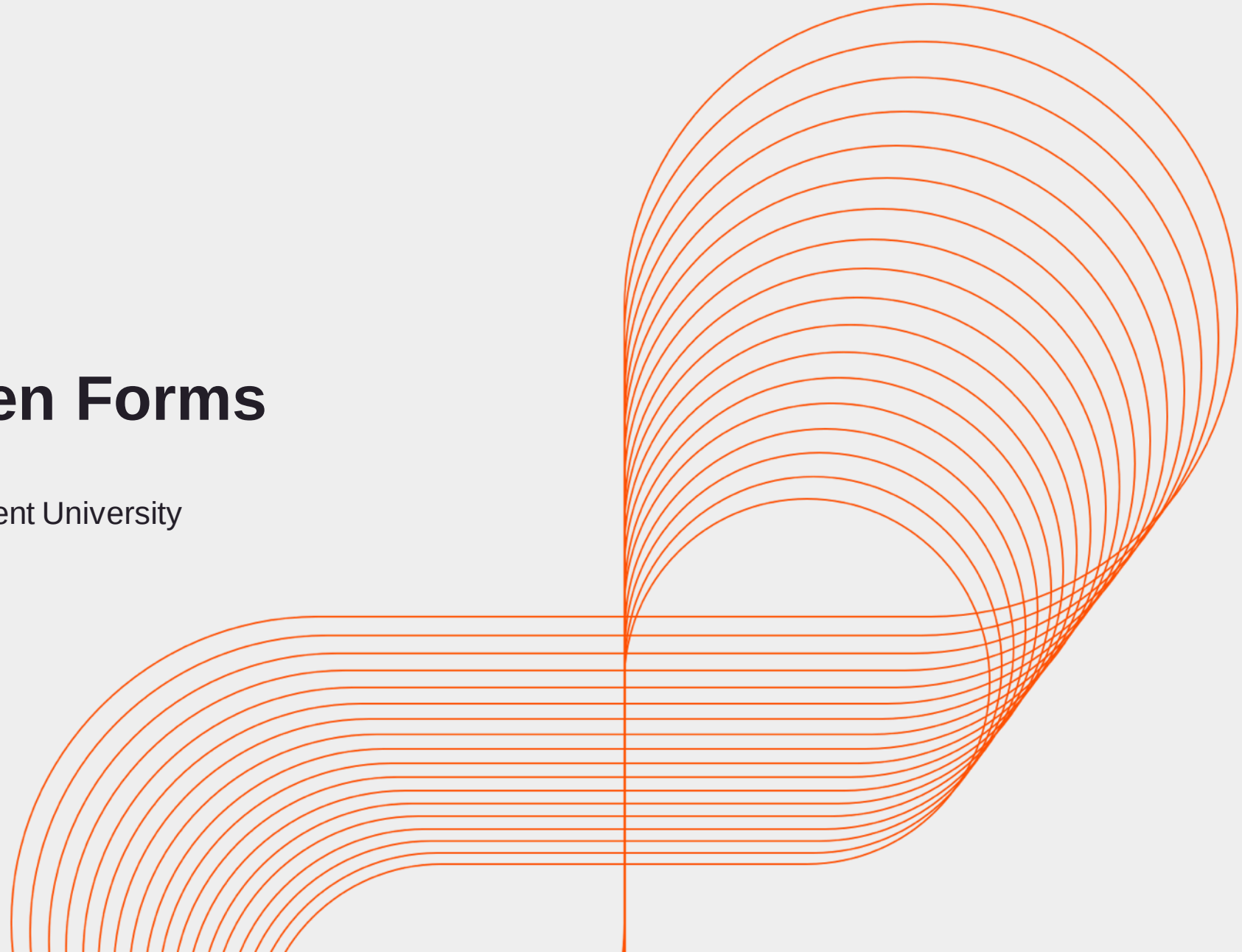
Persistent

Template Driven Forms

Persistent Interactive | Persistent University



Key Learning Points

- Validation Properties exported by ngModel
  - .valid and .invalid
  - .dirty and .pristine
  - .touched and .untouched
- Use of ngForm to check form validity
- Displaying error messages based on the validations.

ngModel directive

- ngModel is an Attribute directive
- Every form control that is registered with ngModel will automatically show up in form.value

```
<form novalidate #userform="ngForm">
<input type="text" name="fname" class="form-control" ngModel >
</form>
```

- With this, we can get the value of **fname** by using **#userform.value.**
- **Applicable for creation forms.**

Mandatory imports

- Import the **FormsModule** in the app.module.ts for template driven forms.

```
import { NgModule }      from '@angular/core';
import { FormsModule }   from '@angular/forms';
```

- Add to the imports Array.

```
@NgModule({
  imports:      [ BrowserModule,FormsModule ],
```

[ngModel] as a model setter

```
<input type="text" class="form-control" [ngModel]=“user.name” >
```

- Set initial values to the form by using one way attribute binding [].
- The actual value of **this.user.name** is never updated upon form changes, this is one-way dataflow.
- Form changes from ngModel are exported onto the respective **#userform.value** properties.
- Defining a name attribute is a requirement when using [(ngModel)] in combination with a form.

[(ngModel)] for Two way Data binding

- Can set initial data from the bound component class, but also update it:

```
<input [(ngModel)]="user.name">
```

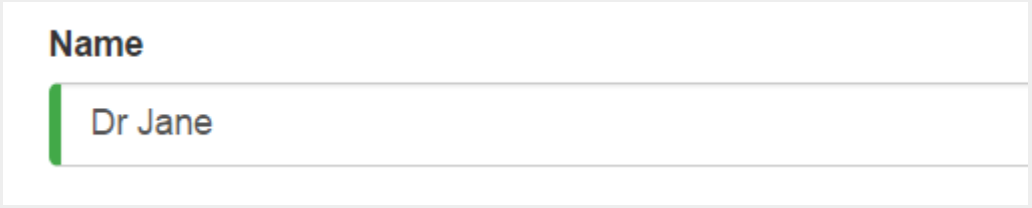
Control states and validity

- **ngModel gives us more features!**
- It updates the control with special Angular CSS classes that reflect the state

State	Class if true	Class if false
Control has been visited	ng-touched	ng-untouched
Control's value has changed	ng-dirty	ng-pristine
Control's value is valid	ng-valid	ng-invalid

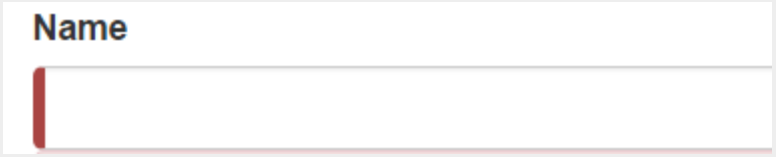
Visual effects of validation

- Green left border for valid values which are valid.




```
.ng-valid:not(form)  {
  border-left: 5px solid
#42A948;
}
```

- Red left border for invalid values which are invalid.




```
.ng-invalid:not(form)  {
  border-left: 5px solid
#a94442;
}
```


Persistent

NgModelGroup

- To semantically group form controls
- Name
  - Firstname
  - Lastname

```
<fieldset ngModelGroup="name">
  <label>Firstname:</label>
  <input type="text" name="firstname" ngModel>

  <label>Lastname:</label>
  <input type="text" name="lastname" ngModel>
</fieldset>
```

- A form is also just a control group.

How to get validation states ?

- **NgModel also offers the validation states for a given form control.**
- Adding a template reference variable like ‘spy’ will only help to fetch native properties of an HTML element.
- To fetch validation properties in Angular context, assign it to ngModel.

```
<input type=“text” #name=“ngModel” >
```

- name.valid
- name.dirty
- name.touched
- name.errors (name.errors?.required , name.errors?.minlength)

Display Error Messages

- The ngModel is exported via the #username.

```
<input type="text” #username ="ngModel" name="name"
[(ngModel)]="user.name" required>
```

- Use the #username to check if the input is valid or not.

```
<div [hidden]="username.valid || username.pristine>
        Username is required
    </div>
```

Exact validation error with Error Object

- Form control with minlength, maxlength and required criteria.

```
<input type="text" required name="contact" #contact="ngModel"
[ngModel]="user.phone" minlength="5" maxlength="10"  >
```

- Display errors with Error Object

```
<div *ngIf="contact.errors?.required" > Contact is required </div>
```

```
<div *ngIf="contact.errors?.minlength" > Contact has to be min 5 chars</div>
```

ngSubmit

- ‘submit’ event is the default event fired when a form is submitted.
- However it causes an actual http post request.
- Since it is a SPA, the post request would be made via Ajax.
- Hence there is **ngSubmit**
- It ensures that the form doesn’t submit.

```
<form #userform="ngForm" (ngSubmit)="postForm()” >
```

- Reset the form to re-set the validation flags as well. **userForm.reset()**

Binding ngForm

- Angular comes with a directive **ngForm** that matches the <form> selector.

```
<form novalidate #userform="ngForm">
```

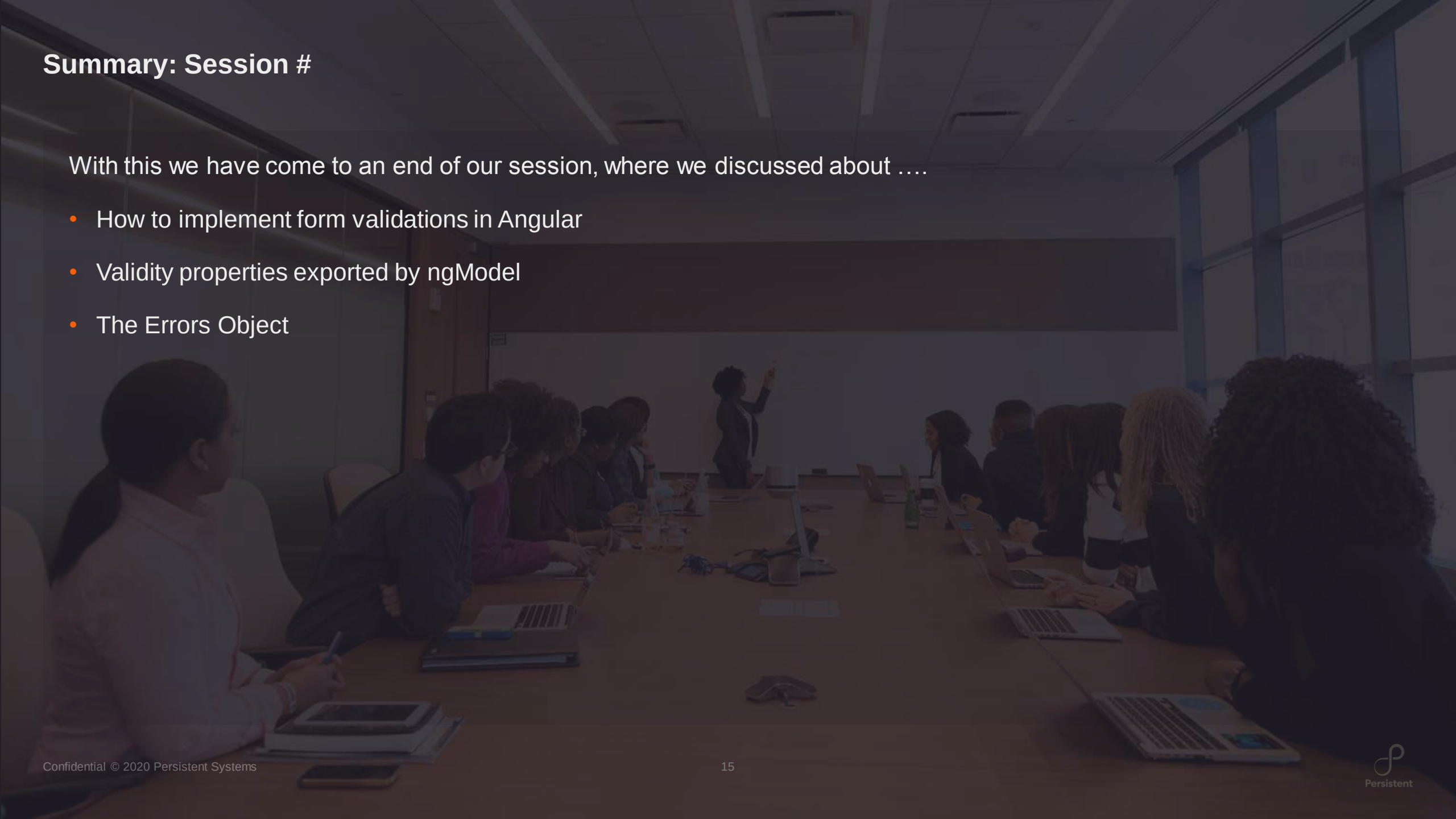
- It provides us information about the current state of the form.
  - A JSON representation of the form value
  - Validity of the form
- The form value can be passed to the postForm().

```
<form novalidate #userform="ngForm“ (ngSubmit)=“postForm(userform.value)”>
```

Summary: Session

With this we have come to an end of our session, where we discussed about ….

- How to implement form validations in Angular
- Validity properties exported by ngModel
- The Errors Object



Appendix

References

Thank You

Reference Links

- Template Driven forms
  - https://toddmotto.com/angular-2-forms-template-driven#template-driven-error-validation
  - http://blog.thoughtram.io/angular/2016/03/21/template-driven-forms-in-angular-2.html
- Template driven vs Model driven
  - http://blog.angular-university.io/introduction-to-angular-2-forms-template-driven-vs-model-driven/
- Custom Validation
  - https://scotch.io/tutorials/how-to-implement-a-custom-validator-directive-confirm-password-in-angular-2
- https://scotch.io/tutorials/using-angular-2s-template-driven-forms
- https://blog.thoughtram.io/angular/2016/10/13/two-way-data-binding-in-angular-2.html
- https://dzone.com/articles/custom-validators-in-template-driven-angular-2-for

Key Contacts :

Persistent University :

- Archana Ghatigar
  archana_ghatigar@persistent.co.in

Persistent

Persistent

Thank you!

Persistent Interactive | Persistent University